

Relatório da Atividade 2 – Etapa 5

Deteção de Intrusões em Rede CAN

Álvaro C. Negromonte¹, Victoria Pessoa Barbosa Matos¹
Pedro C. Guimarães Rodrigues¹

¹Centro de Informática – Universidade Federal de Pernambuco (CIn/UFPE)
IF747 – Redes Automotivas – 2026.1
Recife – PE – Brasil
[acn3, vpbm, pcgr]@cin.ufpe.br

Resumo. Este relatório apresenta a avaliação do módulo de detecção de intrusões `id_time` no ambiente Yes, CARLA CAN. Foram comparadas duas versões do detector: a versão original, que apresentou falsos positivos no cenário normal, e a versão melhorada, que normaliza IDs CAN e calcula anomalias sobre intervalos entre mensagens. Nos logs capturados, a versão melhorada reduziu os falsos positivos para zero e detectou 527 intrusões no ID `0x604` durante o ataque de *spoofing* do freio de mão.

1 Introdução

A Etapa 5 tem como objetivo executar um mecanismo de detecção de intrusões em paralelo com um dos ataques realizados anteriormente e avaliar sua eficácia. Para isso, foi utilizado o detector `id_time`, que compara o comportamento temporal das mensagens CAN recebidas com estatísticas de referência previamente calculadas.

O ataque escolhido para a avaliação foi o *spoofing* da função `hand_brake`. Essa escolha foi feita porque, nas etapas anteriores, esse ataque teve impacto veicular claro no CARLA e também produziu alteração quantitativa expressiva no tráfego do ID `0x604`.

2 Configuração Experimental

O ambiente foi executado com o DBC padrão `data/carla.dbc`, mantendo a compatibilidade com os IDs usados pelo módulo de ataques. O ambiente foi inicializado com:

```
./1_up_environment.sh --dbc data/carla.dbc
```

Em seguida, foram capturados dois logs: um cenário normal, sem ataque, e um cenário com ataque de freio de mão. A captura do cenário normal foi feita com:

```
candump -l vcan0 | tee logs/etapa5_normal.log
```

No cenário de ataque, o IDS foi executado em paralelo com o ataque:

```
conda run --no-capture-output -n n4s_env python3  
→ intrusion_detection_module.py \  
--detector id_time
```

```
conda run --no-capture-output -n n4s_env python3
→ cyberattacks_module.py \
  --feature hand_brake \
  --period 0.05
```

Para reproduzir a análise de forma offline, foi utilizado o script de avaliação incluído no diretório data:

```
conda run --no-capture-output -n n4s_env python
→ data/id_time_detection_analyzer.py \
  --normal logs/etapa5_normal.log \
  --attack logs/etapa5_handbrake_attack.log \
  --output-dir data/etapa5_detection \
  --plots \
  --compare-versions
```

3 Ajuste no Detector

Durante a preparação da Etapa 5, foram identificadas duas inconsistências no detector original. A primeira estava relacionada ao formato dos IDs CAN: o arquivo de estatísticas usa IDs no formato 604, enquanto o detector comparava as mensagens recebidas com strings no formato 0x604. Assim, IDs conhecidos eram classificados incorretamente como desconhecidos.

A segunda inconsistência estava na variável usada para a análise temporal. O baseline descreve os intervalos entre mensagens, mas o detector calculava estatística sobre timestamps absolutos. Isso não representa corretamente o período de chegada das mensagens CAN.

O detector foi ajustado para normalizar os IDs CAN e calcular a média e o desvio padrão dos intervalos entre mensagens recentes. Um alerta é emitido quando a média ou a variabilidade observada se afastam do baseline. Essa alteração mantém a proposta original do algoritmo `id_time`, mas faz a comparação sobre a métrica correta.

4 Resultados

A Tabela 1 resume a comparação entre a versão original e a versão melhorada do detector.

Tabela 1: Comparação entre a versão original e a versão melhorada do detector.

Versão	Normal	Ataque	Principal ID
Original	2818	3026	Vários IDs
Melhorada	0	527	0x604

Na versão original, o detector gerou 2818 alertas no cenário normal. Como esse cenário não continha ataque, esses alertas correspondem a falsos positivos. A taxa observada foi:

$$\frac{2818}{2818} = 100\% \quad (1)$$

Após a melhoria, o detector não gerou alertas no cenário normal:

$$\frac{0}{2818} = 0\% \quad (2)$$

Durante o ataque, a versão melhorada detectou 527 intrusões no ID 0x604. Esse resultado é coerente com o comportamento esperado, pois o ataque injeta mensagens do freio de mão com período de 0,05 s, menor que o período normal observado para esse ID.

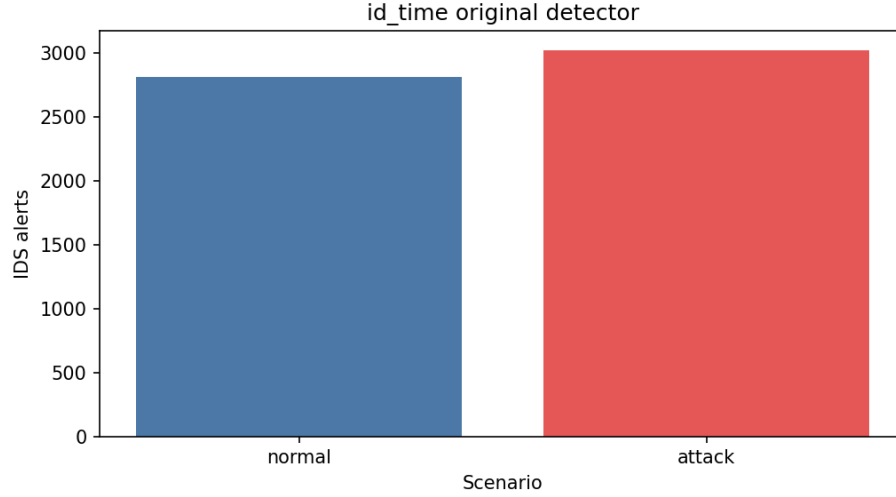


Figura 1: Alertas da versão original do detector nos cenários normal e com ataque.

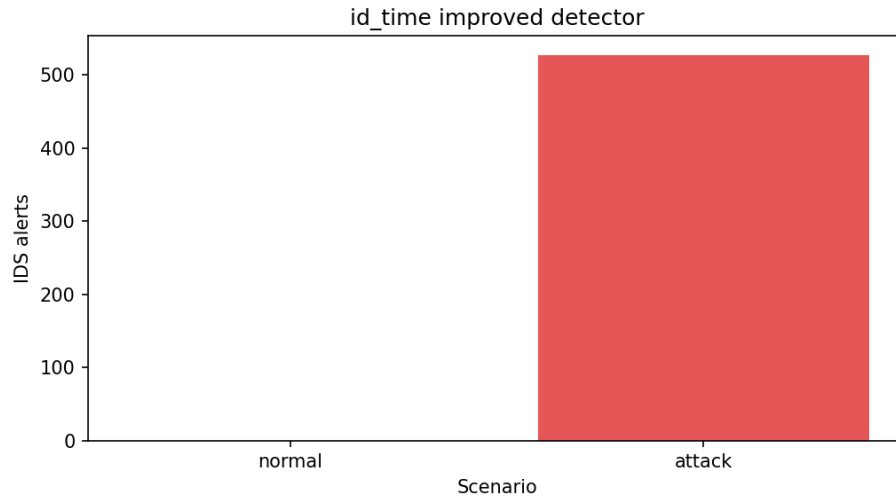


Figura 2: Alertas da versão melhorada do detector nos cenários normal e com ataque.

5 Discussão

Os resultados mostram que a versão original do detector não era adequada para avaliar o cenário experimental, pois gerava falsos positivos mesmo na ausência de ataques. O principal motivo era a incompatibilidade de representação dos IDs CAN. Como o baseline

armazenava IDs como 604 e o detector recebia IDs como 0x604, mensagens legítimas eram tratadas como desconhecidas.

Na versão melhorada, a normalização dos IDs eliminou esse problema. Além disso, o cálculo temporal passou a usar intervalos entre mensagens, e não timestamps absolutos. Com isso, o tráfego normal deixou de ser classificado como intrusão, enquanto o ataque de *spoofing* do freio de mão continuou sendo detectado no ID correto.

O resultado também mostra uma limitação do método: ele é eficaz quando o ataque altera a periodicidade das mensagens, mas pode ser insuficiente contra ataques que preservam o período esperado e modificam apenas o payload.

6 Proposta de Melhoria

Uma melhoria adicional para o algoritmo seria combinar a detecção temporal com validação semântica baseada no DBC. Além de monitorar média e desvio do período por ID, o detector poderia validar o payload decodificado e as transições de estado esperadas para cada função veicular.

Por exemplo, para o freio de mão, o detector poderia observar não apenas a frequência do ID 0x604, mas também transições repetidas ou persistentes para o valor ativo. Para marcha ré, poderia validar se a mudança de estado ocorre de forma compatível com velocidade, marcha e comandos recentes. Essa combinação reduziria falsos negativos contra ataques que imitam o período normal, mas alteram o conteúdo da mensagem.

Também seria recomendável registrar os alertas em CSV ou JSON durante a execução ao vivo, incluindo timestamp, ID, tipo de alerta e valor observado. Isso facilitaria a auditoria dos experimentos e a geração automática de gráficos para relatórios.

7 Conclusão

A Etapa 5 demonstrou a importância de validar o próprio detector antes de avaliar ataques. A versão original apresentou falsos positivos no cenário normal, enquanto a versão melhorada reduziu a taxa de falsos positivos para zero e manteve a detecção do ataque *hand_brake* no ID 0x604. Os resultados indicam que a detecção temporal é útil contra ataques que alteram a frequência das mensagens, mas deve ser combinada com validação de payload e regras de estado para cobrir ataques mais discretos.